

devd/README.md at master · cortesi/devd · GitHub

github.com/cortesi/devd

devd: a local webserver for developers

```
> devd -o ./rendered
11:16:39: Listening on http://devd.io:8001 (127.0.0.1:8001)
11:16:39: GET /
<- 200 OK (10850 bytes)
11:16:39: GET /nonexistent
<- 404 Not Found
11:16:39: GET /css04-syntax.css
<- 200 OK (7446 bytes)
11:16:39: GET /css01-blogstyle-9bc853f1.css
<- 200 OK (116685 bytes)
```

Install

Go to the [releases page](#), download the package for your OS, and copy the binary to somewhere on your PATH.

If you have a working Go installation, you can also say

Quick start

Serve the current directory, open it in the browser (**-o**), and livereload when files change (**-l**):

```
devd -ol .
```

Reverse proxy to <http://localhost:8080>, and livereload when any file in the **src** directory changes:

```
devd -w ./src http://localhost:8080
```

Using devd with modd

[Modd](#) is devd's sister project - a dev tool that runs commands and manages daemons in response to filesystem changes. Devd can be used with modd to rebuild a project and reload the browser when filesystem changes are detected.

Here's a quick example of a simple *modd.conf* file to illustrate.

```
src/** {
  prep: render ./src ./rendered
}

rendered/*.css ./rendered/*.html {
  daemon: devd -m ./rendered
}
```

The first block runs the *render* script whenever anything in the *src* directory changes. The second block starts a devd instance, and triggers livereload with a signal whenever a *.css* or *.html* file in the *rendered* directory changes.

See the [modd](#) project page for details.

Features

Cross-platform and self-contained

Devd is a single statically compiled binary with no external dependencies, and is released for macOS, Linux and Windows. Don't want to install Node or Python in that light-weight Docker instance you're hacking in? Just copy over the devd binary and be done with it.

Designed for the terminal

This means no config file, no daemonization, and logs that are designed to be read in the terminal by a developer. Logs are colorized and log entries span multiple lines. Devd's logs are detailed, warn about corner cases that other daemons ignore, and can optionally include things like detailed timing information and full headers.

Convenient

To make quickly firing up an instance as simple as possible, devd automatically chooses an open port to run on (unless it's specified), and can open a browser window pointing to the daemon root for you (the **-o** flag in the example above). It also has utility features like the **-s** flag, which auto-generates a self-signed certificate for devd, stores it in `~/.devd.certs` and enables TLS all in one step.

Livereload

When livereload is enabled, devd injects a small script into HTML pages, just before the closing *head* tag. The script listens for change notifications over a websocket connection, and reloads resources as needed. No browser addon is required, and livereload works even for reverse proxied apps. If only changes to CSS files are seen, devd will only reload external CSS resources, otherwise a full page reload is done. This serves the current directory with livereload enabled:

```
devd -l .
```

You can also trigger livereload for files that are not being served, letting you reload reverse proxied applications when source files change. So, this command watches the *src* directory tree, and reverse proxies to a locally running application:

```
devd -w ./src http://localhost:8888
```

The **-x** flag excludes files from triggering livereload based on a `.`. The following command disables livereload for all files with the `.less` extension:

```
devd -x "**.less" -l .
```

When livereload is enabled (with the **-L**, **-l** or **-w** flags), devd responds to a SIGHUP by issuing a livereload notice to all connected browsers. This allows external tools, like devd's sister project **modd**, to trigger livereload. If livereload is not enabled, SIGHUP causes the daemon to exit.

The closing *head* tag must be found within the first 30kb of the remote file, otherwise livereload is disabled for the file.

Reverse proxy + static file server + flexible routing

Modern apps tend to be collections of web servers, and devd caters for this with flexible reverse proxying. You can use devd to overlay a set of services on a single domain, add livereload to services that don't natively support it, add throttling and latency simulation to existing services, and so forth.

Here's a more complicated example showing how all this ties together - it overlays two applications and a tree of static files. Livereload is enabled for the static files (**-l**) and also triggered whenever source files for reverse proxied apps change (**-w**):

```
devd -l \  
-w ./src/ \  
/=http://localhost:8888 \  
/api=http://localhost:8889 \  
/static=./assets
```

This is compact but powerful enough to cater for most use cases.

Light-weight virtual hosting

Devd uses a dedicated domain - **localhost** - to do simple virtual hosting. This domain and all its subdomains resolve to 127.0.0.1, which we use to set up virtual hosting without any changes to */etc/hosts* or other local configuration. Route specifications that don't start with a leading */* are taken to be subdomains of **localhost**. So, the following command serves a static site from localhost, and reverse proxies a locally running app on api.localhost:

```
devd ./static api=http://localhost:8888
```

Latency and bandwidth simulation

Want to know what it's like to use your fancy 5mb HTML5 app from a mobile phone in Botswana? Look up the bandwidth and latency [here](#), and invoke devd like so (making sure to convert from kilobits per second to kilobytes per second and account for the location of your server):

```
devd -d 114 -u 51 -n 275 .
```

Devd tries to be reasonably accurate in simulating bandwidth and latency - it uses a token bucket implementation for throttling, properly handles concurrent requests, and chunks traffic up so data flow is smooth.

Routes

The devd command takes one or more route specifications as arguments. Routes have the basic format **root=endpoint**. Roots can be fixed, like *"/favicon.ico"*, or subtrees, like *"/images/"* (note the trailing slash). Endpoints can be filesystem paths or URLs to upstream HTTP servers.

Here's a route that serves the directory *./static* under */assets* on the server:

```
/assets=./static
```

To use a **localhost** subdomain (which will resolve to 127.0.0.1), just add it to the the front of the root specification. We recognize subdomains by the fact that they don't start with a leading */*. So, this route serves the **/static** directory under **static.localhost/assets**:

```
static/assets=./static
```

Reverse proxy specifications are similar, but the endpoint specification is a URL. The following serves a local URL from the root **app.localhost/login**:

```
app/login=http://localhost:8888
```

If the **root** specification is omitted, it is assumed to be */*, i.e. a pattern matching all paths. So, a simple directory specification serves the directory tree directly under **localhost**:

```
devd ./static
```

Similarly, a simple reverse proxy can be started like this:

```
devd http://localhost:8888
```

There is also a shortcut for reverse proxying to localhost:

```
devd :8888
```

Serving default content for files not found

The **--notfound** flag can be passed multiple times, and specifies a set of routes that are consulted when a requested file is not found by the static file server. The basic syntax is **root=path**, where **root** has the same semantics as route specification. As with routes, the **root=** component is optional, and if absent is taken to be equal to */*. The **path** is always relative to the static directory being served. When it starts with a leading slash (*/*), devd will only look for a replacement file in a single location relative to the root of the tree. Otherwise, it will search for a matching file by joining the specified **path** with all path components up to the root of the tree.

Let's illustrate this with an example. Say we have a */static* directory as follows:

```
./static
├── bar
│   └── index.html
└── index.html
```

We can specify that devd should look for an *index.html* anywhere on the path to the root of the static tree as follows:

```
devd --notfound index.html /static
```

Now, the following happens:

- A request for `/nonexistent.html` returns the contents of `/index.html`
- A request for `/bar/nonexistent.html` returns the contents of `/bar/index.html`
- A request for `/foo/bar/voing/index.html` returns the contents of `/index.html`

We could instead specify an absolute path in the route, in which case the contents of `/index.html` would be returned for all the examples above:

```
devd --notfound /index.html /static
```

Devd won't serve an over-ride page if the expected type of the incoming request doesn't match that of the override specification. We do this by looking at the file extension and expected MIME types of the override and request, defaulting to `text/html` if the type couldn't be positively established. This prevents issues where, for instance, an HTML over-ride page might be served where images are expected.

Excluding files from livereload

The `-x` flag supports the following terms:

Term	Meaning
<code>*</code>	matches any sequence of non-path-separators
<code>**</code>	matches any sequence of characters, including path separators
<code>?</code>	matches any single non-path-separator character
<code>[class]</code>	matches any single non-path-separator character against a class of characters
<code>{alt1, ...}</code>	matches a sequence of characters if one of the comma-separated alternatives matches

Any character with a special meaning can be escaped with a backslash (`\`). Character classes support the following:

Class	Meaning
<code>[abc]</code>	matches any single character within the set
<code>[a-z]</code>	matches any single character in the range
<code>[^class]</code>	matches any single character which does <i>not</i> match the class

About reverse proxying

Devd does not validate upstream SSL certificates when reverse proxying. For our use case, development servers will usually be running locally, often with self-signed certificates for testing. You shouldn't use devd in cases where upstream cert validation matters.

The *X-Forwarded-Host* and *X-Forwarded-Proto* headers are set to the devd server's address and protocol for reverse proxied traffic. You might need to enable support for this in your application for redirects and the like to work correctly.

Development

The scripts used to build this package for distribution can be found [here](#). External packages are vendored using [dep](#).